



# UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

*Facultad de Ciencias*

---

Compiladores

Analizador léxico completo de MINIC

---

## Práctica 2

*Presenta:*

Ramírez Juárez María Fernanda - 321204747

Rojo Peña Manuel Ianluck - 118005762

*Profesor:*

Pablo Martínez Yessica Janeth

*Ayudante de laboratorio:*

Salazar González Pedro Yamil

Grupo: 7010, 2027-1

*Fecha de entrega:*

11 de septiembre, 2026

## Objetivo

En esta práctica nuestro objetivo fue extender el analizador léxico de MINIC para que pudiera reconocer construcciones más complejas que un solo símbolo, puesto que, un lenguaje de programación se compone de mucho más que únicamente símbolos de un solo carácter como `=`, `+` o `;`, también se manejan operadores compuestos, palabras reservadas e identificadores.

Concretamente, el objetivo es incorporar:

1. Conservar las funcionalidades ya implementadas en la Práctica 1
2. Reconocer identificadores
3. Distinguir entre identificadores, palabras reservadas (literales booleanos, condicionales)
4. Reconocer operadores compuestos (`==`, `!=`, `<=`, `>=`, `&&`, `||`)
5. Reconocer e ignorar comentarios de una línea
6. Detectar caracteres no válidos y generarlos como `TOKEN_ERROR` con su posición
7. Recuperarnos después de un error, sin detener la ejecución
8. Generar correctamente el `TOKEN_EOF` al final del archivo

Además, tratamos de que el programa siguiera siendo robusto ante cadenas de longitud arbitraria (identificadores o enteros muy largos) y que el flujo de lectura del archivo se mantuviera claro y fácil de seguir. Al retomar la Práctica 1 notamos que, entre más casos añadíamos al ciclo principal, más difícil se volvía leer y de seguir, por lo que en esta práctica decidimos modularizar cada implementación de nuevo código en vez de simplemente apilar más condicionales por caso dentro del mismo ciclo.

## Evolución respecto de la Práctica 1

Sin entrar mucho detalle, ya que lo describiremos más profundamente en la siguiente sección, lo que hicimos fue hacer la función `lexer_scan` mucho más legible.

### Módulos modificados o agregados

- `lexer.h`: se agregó la estructura `Lexer`, que concentra todo el estado que antes manejábamos como variables y parámetros sueltos (`file`, `line`, `column`, `last_was_cr`, etc.), con un campo nuevo (`current`), que explicaremos más adelante, pero de manera rápida, nos indica el carácter actual.
- `lexer.c`: se reescribió casi por completo, la función `simple_token_type` se convirtió en `scan_simple` con un alcance más reducido, `advance_position` se fusionó dentro de `lexer_advance`, y se agregaron las funciones nuevas como `lexer_init`, `lexer_check`, `emit_token`, `scan_operator`, `scan_integer`, `scan_keyword` y `skip_comment_content`.

- `keywords.h` y `keywords.c`: son un nuevo módulo, definimos la estructura `Keyword` y la función `lookup_keyword`, que traduce un lexema a su `TokenType` si corresponde a una palabra reservada.
- `buffer_manage.h` y `buffer_manage.c`: módulo nuevo, implementamos un buffer de texto dinámico (`TextBuffer`) que usamos para acumular lexemas de longitud variable no conocida de antemano (enteros, identificadores y palabras reservadas).

## Cambios en la representación de tokens

En la Práctica 1, solo emparejábamos los símbolos simples como `PLUS`, `MINUS`, `STAR`, `SLASH`, `ASSIGN`, `LESS`, `GREATER`, `LPAREN`, `RPAREN`, `LBRACE`, `RBRACE` y `SEMICOLON`; tokens de un solo carácter, sin distinguir todavía entre un `=` suelto y un `==`, desaprovechando todos los demás tokens que en su enumeración ya venían definidos como `EQUAL`, `NOT_EQUAL`, `GREATER_EQUAL`, etc.

```
1  /* */
2  typedef enum {
3      TOKEN_EOF ,
4      ERROR ,
5      IDENTIFIER ,
6      INTEGER ,
7      INT ,
8      BOOL ,
9      IF ,
10     ELSE ,
11     WHILE ,
12     PRINT ,
13     TRUE ,
14     FALSE ,
15     PLUS ,
16     MINUS ,
17     STAR ,
18     SLASH ,
19     ASSIGN ,
20     EQUAL ,
21     NOT_EQUAL ,
22     LESS ,
23     LESS_EQUAL ,
24     GREATER ,
25     GREATER_EQUAL ,
26     AND ,
27     OR ,
28     LPAREN ,
29     RPAREN ,
30     LBRACE ,
31     RBRACE ,
32     SEMICOLON
33 } TokenType;
```

Código 1: Enumeración completa de `TokenType`, definida desde la Práctica 1 en `token.h`.

Todos ellos se manejaban en lo que era la función `simple_token_type`, que ahora (sin entrar en mucho detalle), la hemos refactorizado y cambiado el nombre a `scan_simple`, donde ahora

solo leemos los caracteres que son únicamente simples, es decir que no pueden (al menos en este contexto de MINIC ) llegar a ser un operador compuesto:

```
1  /* Escaneamos un caracter para verificar que tipo de simbolo simple es */
2  static int
3  scan_simple(int c,
4             TokenType *type)
5  {
6      if (type == NULL)
7          return 0;
8
9      switch (c) {
10         case '+':
11             *type = PLUS;
12             break;
13         case '-':
14             *type = MINUS;
15             break;
16         case '*':
17             *type = STAR;
18             break;
19         case '(':
20             *type = LPAREN;
21             break;
22         case ')':
23             *type = RPAREN;
24             break;
25         case '{':
26             *type = LBRACE;
27             break;
28         case '}':
29             *type = RBRACE;
30             break;
31         case ';':
32             *type = SEMICOLON;
33             break;
34         default:
35             return 0;
36     }
37
38     return 1;
39 }
```

Código 2: `scan_simple` de la Práctica 1 adaptado a las nuevas necesidades.

Sigue sin ser la gran cosa, pues solo es un `switch` para emparejar el caso de cada simbolo con su token, nótese que, como mencionamos, ya no están los casos para aquellos símbolos que pueden derivar en otros operadores.

## Correcciones efectuadas a la implementación anterior

La corrección más relevante fue reemplazar el tamaño fijo del buffer que usábamos en la Práctica 1 para leer enteros. En ese momento, por falta de tiempo, no retomamos la lógica de buffer dinámico que habíamos visto en una de las primeras clases de laboratorio, y simplemente reservamos un arreglo de 64 bytes en la pila:

```
1 if (isdigit(c)) {
2     char buffer[64];
3     size_t len = 0;
4     buffer[len++] = (char)c;
5
6     int next;
7     while ((next = fgetc(file)) != EOF && isdigit(next)) {
8         if (len + 1 >= sizeof(buffer)) {
9             fprintf(stderr, "Error: numero demasiado largo.\n ");
10            return 2;
11        }
12        buffer[len++] = (char)next;
13        advance_position(next, &line, &column, &last_was_cr);
14    }
15    buffer[len] = '\0';
16    ...
17 }
```

Código 3: Buffer de tamaño fijo usado en la Práctica 1 (`lexer-p1.c`).

Esto funcionaba para los casos de prueba de la Práctica 1, pero era una limitante considerable, un identificador o un entero legítimo más largo de 63 caracteres haría fallar al programa con un error que no tiene nada que ver con un error léxico real del programa fuente. En esta práctica sustituimos por completo esta lógica por el módulo `buffer_manage`, que describimos a detalle más adelante, y que tanto `scan_integer` como `scan_keyword` reutilizan sin duplicar código entre sí.

## Refactorizaciones relevantes y su justificación

- **`simple_token_type` → `scan_simple`.** Le quitamos los símbolos `=`, `<`, `>` y `/`, porque ahora estos pueden formar parte de un operador compuesto o de un comentario, y decidir eso requiere mirar el siguiente carácter antes de emitir un token. Dejar estos símbolos en `scan_simple` habría significado alargar y añadir complejidad innecesaria a este función.
- **`advance_position` → `lexer_advance`.** La función conserva la misma lógica de actualización de línea y columna, pero ahora es un método de la estructura `Lexer` y además es la única función responsable de leer un carácter nuevo del archivo. Esto evita que la actualización de posición y la lectura del archivo se hagan en dos lugares distintos (que fue, adelantando un poco, la causa de varios de los problemas que describimos en la sección de Problemas encontrados).
- **emit\_token como función auxiliar nueva.** En la Práctica 1, el patrón `token_init` - `token_print` - `token_destroy` se repetía en cada rama del `if`. Lo extrajimos a una sola función para que cada rama de `lexer_scan` se reduzca a una sola línea, y para que un futuro cambio en cómo se emite un token (por ejemplo, un formato de salida distinto) solo tenga que ajustarse en un solo lugar.
- **La estructura `Lexer` en sí misma.** Nuestra motivación principal era que las funciones auxiliares (`scan_integer`, `scan_keyword`, `scan_operator`) necesitaban acceso al archivo y a la posición actual, y no queríamos pasarles cuatro parámetros sueltos a cada una (algo que sí hacíamos en un borrador intermedio). Agrupar todo en un solo `struct` y pasar un

solo apuntador resuelve esto, y de paso deja espacio para el mecanismo de *mirar-antes-de-consumir* que terminamos necesitando.

## Descripción de la solución

Con `simple_token_type` ya reducida a `scan_simple`, `lexer_scan` queda mucho más legible: se limita a mirar el primer carácter de cada posible token y delegar el resto de la lectura a una función auxiliar según el caso.

```
1 /* Estructura con la que podemos llevar un mejor flujo del proceso del lexer */
2 typedef struct {
3     FILE *file;
4     size_t line;
5     size_t column;
6     int last_was_cr;
7     int current;
8 } Lexer;
```

Código 4: La estructura `Lexer`, definida en `lexer.h`.

Cada campo tiene un propósito concreto:

- `file`: el `FILE*` del que estamos leyendo, para no tener que pasarlo aparte a cada función auxiliar.
- `line` / `column`: la posición actual dentro del archivo, por invariante de diseño, `line` y `column` siempre corresponden a la posición del carácter guardado en `current` (no a la del último carácter leído del disco, que puede ser distinto, como lo explicaremos más adelante).
- `last_was_cr`: la bandera con la que recordamos si el carácter inmediatamente anterior fue un `\r`. La necesitamos para tratar la secuencia `\r\n` como un único salto de línea, si ya contamos el salto de línea al procesar el `\r`, el `\n` que le sigue no debe contarse una segunda vez.
- `current`: el carácter que el lexer ya sacó del archivo pero que todavía no se ha consumido como parte de ningún token. Es la pieza que nos permite *mirar* el siguiente carácter sin tener que procesarlo todavía, y es justo lo que necesitábamos para decidir, por ejemplo, si un `=` suelto es `ASSIGN` o el primer carácter de `EQUAL`.

`lexer_init` deja el `Lexer` listo para empezar, precargando el primer carácter del archivo en `current`:

```
1 /* Inicializamos la estructura lexer para empezar a leer el archivo */
2 static void
3 lexer_init(Lexer *lex,
4            FILE *file)
5 {
6     lex->file = file;
7     lex->line = 1;
8     lex->column = 0;
9     lex->last_was_cr = 0;
10    lex->current = fgetc(file);
11 }
```

Código 5: Función `lexer_init`.

`lexer_check` simplemente devuelve ese caracter ya precargado, sin tocar el archivo ni la posición:

```
1 /* Devuelve el caracter aun no consumido, sin leer del archivo */
2 static int
3 lexer_check(Lexer *lex)
4 {
5     return lex->current;
6 }
```

Código 6: Función `lexer_check`.

Y `lexer_advance` es la única función que de verdad consume un caracter, toma lo que hay en `current`, actualiza línea y columna según corresponda, y precarga en `current` el siguiente carácter del archivo para que quede listo para la próxima llamada.

```
1 /* Leemos un caracter del archivo y actualiza linea/columna segun corresponda
   */
2 static int
3 lexer_advance(Lexer *lex)
4 {
5     int c = lexer_check(lex);
6
7     if (c != EOF) {
8         if (c == '\r'){
9             (lex->line)++;
10            lex->column = 0;
11            lex->last_was_cr = 1;
12        } else if (c == '\n') {
13            if (lex->last_was_cr) {
14                lex->last_was_cr = 0;
15            } else {
16                (lex->line)++;
17                lex->column = 0;
18            }
19        } else {
20            (lex->column)++;
21            lex->last_was_cr = 0;
22        }
23        lex->current = fgetc(lex->file);
24    }
25
26    return c;
27 }
```

Código 7: Función `lexer_advance`.

Con esto en mente y ya establecido, `lexer_scan` queda así:

```
1 /* Funcion principal que comienza el escaneo de un archivo para ser procesado
   por el lexer */
2 int
3 lexer_scan(FILE *file)
4 {
5     int c;
6     Lexer lex;
```



```
7
8     if (file == NULL)
9         return 2;
10
11     lexer_init(&lex, file);
12
13     while ((c = lexer_check(&lex)) != EOF) {
14         size_t token_line = lex.line;
15         size_t token_column = lex.column;
16         TokenType type;
17
18         lexer_advance(&lex);
19
20         if (is_ignored_space(c))
21             continue;
22
23         //Comentarios
24         if (c == '/') {
25             if (lexer_check(&lex) == '/') {
26                 lexer_advance(&lex);
27                 skip_comment_content(&lex);
28                 continue;
29             }
30             if (!emit_token(SLASH, "/", token_line, token_column))
31                 return 2;
32             continue;
33         }
34
35         //Operadores
36         if (strchr("<=>!&|", c) != NULL) {
37             char lexeme[3];
38             type = scan_operator(&lex, c, lexeme);
39             if (!emit_token(type, lexeme, token_line, token_column))
40                 return 2;
41             continue;
42         }
43
44         //Numeros enteros
45         if (isdigit(c)) {
46             if (!scan_integer(&lex, c, token_line, token_column))
47                 return 2;
48             continue;
49         }
50
51         //Identificadores y palabras reservadas
52         if (isalpha(c) || c == '_') {
53             if (!scan_keyword(&lex, c, token_line, token_column))
54                 return 2;
55             continue;
56         }
57
58         //Simples
59         if (scan_simple(c, &type)) {
60             char lexeme[2] = {(char)c, '\0'};
61             if (!emit_token(type, lexeme, token_line, token_column))
62                 return 2;
```

```

63         continue;
64     }
65
66     //Errores lexicos
67     char lexeme[2] = {(char)c, '\0'};
68     if (!emit_token(ERROR, lexeme, token_line, token_column))
69         return 2;
70 }
71
72 if (ferror(file)) {
73     fprintf(stderr, "Error: no se pudo leer el archivo.\n");
74     return 2;
75 }
76
77 if (!emit_token(TOKEN_EOF, "", lex.line, lex.column))
78     return 2;
79
80 return 0;
81 }

```

Código 8: Función principal `lexer_scan`.

Vale la pena que resaltemos el cambio en el ciclo principal, antes leíamos directamente con `fgetc(file)` dentro de la condición del `while`, lo que quería decir que el caracter ya estaba consumido en el momento en que entrábamos al cuerpo del ciclo. Ahora usamos `lexer_check` para *mirar* el caracter (sin consumirlo todavía), guardamos `token_line` y `token_column` con la posición correcta de ese carácter, y solo después llamamos a `lexer_advance` para consumirlo de verdad. Este orden de mirar, guardar posición, y hasta entonces consumir, es precisamente lo que terminamos necesitando para que la posición reportada de cada token fuera la correcta (más detalles en la sección de Problemas encontrados).

Con esta base ya podemos explicar cada uno de los casos que maneja `lexer_scan`.

## Distinción entre SLASH y comentarios

Como `/` ya no forma parte de `scan_simple`, tiene su propio caso dentro de `lexer_scan`, al leer un `/`, miramos (con `lexer_check`, sin consumir) si el caracter que sigue es también un `/`. Si lo es, ambos se consumen y delegamos a `skip_comment_content` el trabajo de ignorar el resto de la línea. Si no lo es, el `/` original se clasifica simplemente como `SLASH`.

```

1  /* Descartamos el contenido de un comentario hasta \n o EOF */
2  static void
3  skip_comment_content(Lexer *lex)
4  {
5      while (lexer_check(lex) != EOF && lexer_check(lex) != '\n')
6          lexer_advance(lex);
7      if (lexer_check(lex) == '\n')
8          lexer_advance(lex);
9  }

```

Código 9: Función `skip_comment_content`.

Aquí no necesitamos ninguna lógica especial de conteo de línea/columna, como cada caracter dentro del comentario se descarta llamando a `lexer_advance` (la misma función que usamos para

cualquier otro caracter del archivo), la posición se actualiza exactamente igual que si esos caracteres formaran parte de un token real, incluyendo el salto de línea final del comentario. Esta es justo una de las ventajas de haber concentrado toda la lógica de avance en una sola función, no tuvimos que reimplementar el conteo de posición para este caso particular.

## Estrategia de anticipación para operadores compuestos

```
1  /*
2  * Comprobamos si =,<,>,!,&,| forman un operador compuesto (==, <=, >=, !=, &&,
3  * o quedan como su version simple.
4  */
5  static TokenType
6  scan_operator(Lexer *lex,
7                int c,
8                char* lexeme)
9  {
10     int next = lexer_check(lex);
11     TokenType type;
12     int consumed_next = 0;
13
14     switch (c) {
15     case '=':
16         if (next == '=') {
17             type = EQUAL;
18             consumed_next = 1;
19         }
20         else
21             type = ASSIGN;
22         break;
23     case '!':
24         if (next == '=') {
25             type = NOT_EQUAL;
26             consumed_next = 1;
27         }
28         else
29             type = ERROR;
30         break;
31     case '<':
32         if (next == '=') {
33             type = LESS_EQUAL;
34             consumed_next = 1;
35         }
36         else
37             type = LESS;
38         break;
39     case '>':
40         if (next == '=') {
41             type = GREATER_EQUAL;
42             consumed_next = 1;
43         }
44         else
45             type = GREATER;
46         break;
47     case '&':
```

```

48     if (next == '&') {
49         type = AND;
50         consumed_next = 1;
51     }
52     else
53         type = ERROR;
54     break;
55 case '|':
56     if (next == '|') {
57         type = OR;
58         consumed_next = 1;
59     }
60     else
61         type = ERROR;
62     break;
63 default:
64     type = ERROR;
65     break;
66 }
67
68 if (consumed_next) {
69     lexer_advance(lex);
70     lexeme[0] = (char)c;
71     lexeme[1] = (char)next;
72     lexeme[2] = '\\0';
73 } else {
74     lexeme[0] = (char)c;
75     lexeme[1] = '\\0';
76 }
77
78 return type;
79 }

```

Código 10: Función `scan_operator`.

A esta función solo se entra si ya sabemos que `c` es uno de `=`, `<`, `>`, `!`, `&`, `|` (eso lo filtra `strchr` en `lexer_scan` antes de llamarla). Dentro, miramos con `lexer_check` el carácter que sigue (sin consumirlo todavía) y decidimos, si forma el operador compuesto correspondiente, consumimos ese segundo carácter con `lexer_advance` y construimos el lexema de dos caracteres; si no, dejamos ese segundo carácter intacto para la siguiente vuelta del ciclo y devolvemos el tipo simple (o `ERROR`, en el caso de `!`, `&` y `|` quedarse solos, ya que en MINIC no definimos ningún operador de un solo carácter para estos símbolos).

## Reconocimiento de identificadores, palabras reservadas y literales booleanos

```

1  /* Escaneamos caracter por caracter en busca de un identificador o palabra
   /* reservada */
2  static int
3  scan_keyword(Lexer *lex,
4               int first_char,
5               size_t token_line,
6               size_t token_column)
7  {
8      TextBuffer buffer;

```

```

9   TokenType kw_type;
10  int status;
11
12  if (!buffer_init(&buffer, 16)) {
13      fprintf(stderr, "Error: no se pudo reservar memoria.\n");
14      return 0;
15  }
16
17  if (!buffer_append(&buffer, (char)first_char)) {
18      fprintf(stderr, "Error: no se pudo reservar memoria.\n");
19      buffer_free(&buffer);
20      return 0;
21  }
22
23  while (lexer_check(lex) != EOF &&
24         (isalnum(lexer_check(lex)) || lexer_check(lex) == '_')) {
25      if (!buffer_append(&buffer, (char)lexer_advance(lex))) {
26          fprintf(stderr, "Error: no se pudo reservar memoria.\n");
27          buffer_free(&buffer);
28          return 0;
29      }
30  }
31
32  if (lookup_keyword(buffer.data, &kw_type))
33      status = emit_token(kw_type, buffer.data, token_line, token_column);
34  else {
35      status = emit_token(IDENTIFIER, buffer.data, token_line, token_column);
36  }
37
38  buffer_free(&buffer);
39  return status;
40 }

```

Código 11: Función scan\_keyword.

A esta función se entra cuando el primer caracter es una letra o un guion bajo (`isalpha(c) || c == '_'`), cumpliendo el patrón `[a-zA-Z][a-zA-Z0-9_]*` del enunciado para ese caso. Lo que implementamos es que vamos acumulando en el `TextBuffer` todo caracter siguiente que sea letra, dígito o guion bajo, sin importar si hay espacios o no alrededor (el ciclo se detiene solo, naturalmente, en cuanto encuentra un caracter que ya no cumple esa condición). Al terminar de leer la cadena completa es cuando decidimos qué token le corresponde, llamamos a `lookup_keyword` pasándole el lexema completo, y si aparece en la tabla de palabras reservadas se emite ese `TokenType` (`INT`, `BOOL`, `IF`, ..., `TRUE`, `FALSE`); si no aparece, se emite como `IDENTIFIER`.

Esto también explica cómo distinguimos identificadores de literales booleanos, no hay ninguna lógica especial para `true` y `false`, los manejamos exactamente igual que cualquier otra palabra reservada, en la misma tabla de `keywords.c`:

```

1  /* Representacion de una palabra reservada para un mapeo adecuado */
2  typedef struct {
3      const char *lexeme;
4      TokenType type;
5  } Keyword;
6

```

```
7 /* Buscamos el lexema con el mapeo anterior y si existe devolvemos su TokenType
   */
8 int lookup_keyword(const char *lexeme, TokenType *type);
```

Código 12: Módulo keywords.h: representación de una palabra reservada.

```
1 /* Mapeo de las palabras reservadas */
2 static const Keyword keywords[] = {
3     {"int", INT},
4     {"bool", BOOL},
5     {"if", IF},
6     {"else", ELSE},
7     {"while", WHILE},
8     {"print", PRINT},
9     {"true", TRUE},
10    {"false", FALSE}
11 };
12
13 /* Buscamos el lexema con el mapeo anterior y si existe devolvemos su TokenType
   */
14 int
15 lookup_keyword(const char *lexeme,
16               TokenType *type)
17 {
18     size_t n = sizeof(keywords) / sizeof(keywords[0]);
19
20     for (size_t i = 0; i < n; i++) {
21         if (strcmp(lexeme, keywords[i].lexeme) == 0) {
22             *type = keywords[i].type;
23             return 1;
24         }
25     }
26
27     return 0;
28 }
```

Código 13: Módulo keywords.c: tabla de palabras reservadas y lookup\_keyword.

lookup\_keyword recorre la tabla con una búsqueda lineal simple (con solo ocho entradas no hace falta nada más elaborado que un strcmp por cada una) y, si encontramos una coincidencia exacta, escribimos el TokenType correspondiente en \*type y regresamos 1; si no encontramos nada, regresamos 0 y scan\_keyword interpreta eso como un identificador.

## Reconocimiento de números enteros

```
1 /* Escaneamos caracter por caracter en busca de un entero */
2 static int
3 scan_integer(Lexer *lex,
4             int first_char,
5             size_t token_line,
6             size_t token_column)
7 {
8     TextBuffer buffer;
```

```
9      int status;
10
11      if (!buffer_init(&buffer, 16)) {
12          fprintf(stderr, "Error: no se pudo reservar memoria.\n");
13          return 0;
14      }
15
16      if (!buffer_append(&buffer, (char)first_char)) {
17          fprintf(stderr, "Error: no se pudo reservar memoria.\n");
18          buffer_free(&buffer);
19          return 0;
20      }
21
22      while (lexer_check(lex) != EOF && isdigit(lexer_check(lex))) {
23          if (!buffer_append(&buffer, (char)lexer_advance(lex))) {
24              fprintf(stderr, "Error: no se pudo reservar memoria.\n");
25              buffer_free(&buffer);
26              return 0;
27          }
28      }
29      status = emit_token(INTEGER, buffer.data, token_line, token_column);
30
31      buffer_free(&buffer);
32      return status;
33 }
```

Código 14: Función `scan_integer`.

El funcionamiento es análogo a `scan_keyword`, pero exigiendo únicamente dígitos (`[0-9]+`). Aquí surgió una discusión interna sobre si debíamos limitar el número de dígitos para que el valor cupiera, por ejemplo, en un `int` de 32 bits. Decidimos que esa validación no le corresponde a esta capa, el enunciado de la práctica solo pide reconocer la secuencia de dígitos como token `INTEGER`, y cualquier límite de rango es responsabilidad de una etapa posterior del compilador (por ejemplo, un análisis semántico), no del lexer.

## Generación de tokens `ERROR` y recuperación

Cualquier caracter que no haya sido reconocido por ninguno de los casos anteriores (comentarios, operadores, enteros, identificadores/palabras reservadas o símbolos simples) cae en la última rama de `lexer_scan`, donde se emite como `ERROR` conservando el caracter problemático como lexema, junto con su línea y columna. Como en esta etapa solo estamos clasificando caracteres y no aplicando ninguna regla sintáctica, no hay ningún motivo para detener la ejecución del programa, basta con emitir el token `ERROR` y seguir con el siguiente caracter en la próxima vuelta del ciclo `while`. Esto es lo que nos permite, por ejemplo, reportar varios caracteres inválidos en un mismo archivo (ver la sección de Resultados y pruebas).

## Generación y posición de `TOKEN_EOF`

Cuando el ciclo principal termina (es decir, `lexer_check` devuelve `EOF`) y no hubo ningún error de lectura (`ferror`), emitimos un único `TOKEN_EOF` usando la posición que en ese momento tienen `lex.line` y `lex.column`. Gracias a que `lexer_advance` actualiza la posición para *cualquier* caracter que consuma (incluyendo espacios en blanco y el contenido de comentarios), la posición final

es siempre la que corresponde inmediatamente después del último carácter procesado del archivo, sin importar si el archivo termina con espacios, con un comentario, o directamente con un token real.

## Administración de memoria relevante

El módulo `buffer_manage` es el que más responsabilidad de memoria concentra en esta práctica. Por simplicidad no incluimos aquí la implementación completa de `buffer_manage.c` (ya que esta fue vista en clase de laboratorio), solo su interfaz:

```
1  /*
2  * Buffer dinámico de texto, usado para acumular lexemas
3  * (identificadores, palabras reservadas, enteros, etc.)
4  * de longitud no conocida de antemano.
5  */
6  typedef struct {
7      char    *data;
8      size_t   length;    /* número de caracteres almacenados, sin contar '\0' */
9      size_t   capacity;  /* tamaño reservado en 'data' */
10 } TextBuffer;
11
12 /*
13 * Inicializa 'buffer' reservando 'initial_capacity' bytes.
14 * Si 'initial_capacity' es 0, se usa 1 como mínimo.
15 * Devuelve 1 en éxito, 0 si malloc falla (buffer queda en estado seguro,
16 * apto para pasarse a buffer_free sin problema).
17 */
18 int
19 buffer_init(TextBuffer *buffer,
20             size_t initial_capacity);
21
22 /*
23 * Agrega el carácter 'c' al final de 'buffer', creciendo la capacidad
24 * (duplicándola) si hace falta. Mantiene 'data' siempre terminado en '\0'.
25 * Devuelve 1 en éxito, 0 si realloc falla o se detecta overflow de
26 * capacidad (en ese caso 'buffer' queda sin modificar y sigue siendo
27 * válido para buffer_free).
28 */
29 int
30 buffer_append(TextBuffer *buffer, char c);
31
32 /*
33 * Libera la memoria de 'buffer' y lo deja en un estado válido
34 * y reutilizable (data = NULL, length = capacity = 0).
35 * Seguro de llamar aunque buffer_init haya fallado.
36 */
37 void
38 buffer_free(TextBuffer *buffer);
```

Código 15: Interfaz de `buffer_manage.h`: buffer dinámico de texto.

De manera sencilla, en `buffer_init` reservamos un mínimo de capacidad (1 byte, si se pide 0) para que `buffer.data` nunca sea un apuntador colgante mientras el buffer exista. En `buffer_append`, cuando ya no hay espacio, duplicamos la capacidad con `realloc`, con una verificación previa contra `SIZE_MAX / 2` para no desbordar el cálculo de la nueva capacidad en el caso (poco probable,



pero posible) de una cadena extremadamente larga, y mantenemos siempre el buffer terminado en `'\0'` para poder pasarlo directo a funciones que esperan una cadena de C. `buffer_free` libera la memoria y dejamos `data` en `NULL` para que sea seguro llamarlo más de una vez o después de una inicialización fallida.

Un punto que nos pareció importante verificar es qué pasa con la memoria del `TextBuffer` una vez que se emite el token, resulta que, tanto `scan_integer` como `scan_keyword` llaman a `buffer_free` justo después de `emit_token`, en la misma función. Esto solo es seguro porque `token_init` (de la Práctica 1) hace su propia copia del lexema en vez de quedarse con el apuntador que le pasamos; si no fuera así, el `Token` terminaría con un apuntador colgante apenas se liberara el `TextBuffer`. Cada ruta de fallo de memoria (`buffer_init` o `buffer_append` devolviendo 0) libera lo que ya se había reservado antes de reportar el error hacia `lexer_scan`, que a su vez responde con el código de salida 2, siguiendo el mismo protocolo de errores que ya usábamos en la Práctica 1.

Y de este modo es como implementamos nuestro código para completar el lexer de MINIC .

## Resultados y pruebas

Recordamos que, como se indica en el enunciado, las pruebas públicas no sustituyen las pruebas diseñadas por el equipo, así que además de correr las que nos dieron, diseñamos casos propios en `tests/unit_tests/` enfocados en los límites de la especificación que consideramos más propensos a esconder errores.

### Pruebas de regresión de la Práctica 1

Corrimos de nuevo toda la batería de pruebas de la Práctica 1 (`make test-p1`) con nuestro nuevo analizador, y las diez pasan sin problema, con una única excepción esperada, nuestra propia prueba adicional de la práctica pasada, `p_double_char`, que codificaba a propósito el comportamiento de la Práctica 1 (donde `==` debía verse como dos tokens `ASSIGN` separados, porque en ese momento el analizador no tenía anticipación de un carácter). Con los operadores compuestos ya implementados, ese comportamiento ya no es el correcto, así que es de esperarse (y hasta deseable) que esta prueba en particular falle ahora:

```
[code/] > make test-unit
FAIL: p_double_char
--- tests/unit_tests/expected/p_double_char.out 2026-09-06 21:27:40.637351644 -0600
+++ build/p_double_char.actual 2026-09-13 09:50:44.367038776 -0600
@@ -1,16 +1,13 @@
 1:0 INTEGER 1
-1:1 ASSIGN =
-1:2 ASSIGN =
+1:1 EQUAL ==
 1:3 INTEGER 2
 1:4 SEMICOLON ;
 2:0 INTEGER 2
-2:1 LESS <
-2:2 ASSIGN =
+2:1 LESS_EQUAL <=
 2:3 INTEGER 3
 2:4 SEMICOLON ;
 3:0 INTEGER 4
-3:1 GREATER >
-3:2 ASSIGN =
+3:1 GREATER_EQUAL >=
 3:3 INTEGER 5
 3:4 SEMICOLON ;
 4:0 TOKEN_EOF
PASS: ut_comment_literal_newline
PASS: ut_compound_ops_and_comments
PASS: ut_dense_no_spaces
PASS: ut_error_recovery_mixed
----
Total: 5 Pass: 4 Fail: 1
make: *** [Makefile:75: test-unit] Error 1
```

Este resultado confirma que el cambio de comportamiento es exactamente el que buscábamos, == ahora se reconoce como un único token EQUAL, <= como LESS\_EQUAL y >= como GREATER\_EQUAL, en vez de como dos tokens simples consecutivos, por lo que retiramos esta prueba obsoleta de para no confundir pruebas futuras.

## Casos límite

La primera prueba propia (ut\_comment\_literal\_newline) verifica dos límites a la vez, que la secuencia literal de dos caracteres \n dentro de un comentario (no un salto de línea real, sino el texto tal cual) no se confunda con el fin del comentario, y que la cadena ifif se reconozca como un único identificador y no como dos tokens IF pegados (es decir, verificamos que aplicamos la coincidencia más larga posible al reconocer identificadores frente a palabras reservadas).

```
//esto es un comentario\n else
while int ifif else
//fin while archivo
```

```
2:0 WHILE while
2:6 INT int
2:10 IDENTIFIER ifif
2:15 ELSE else
3:19 TOKEN_EOF
```

La segunda (ut\_dense\_no\_spaces) comprueba que el analizador no depende de espacios en blanco para separar tokens, como las reglas de identificadores y palabras reservadas se basan en letras, dígitos y guiones bajos, y los operadores y delimitadores son caracteres completamente distintos a esos, el límite entre un token y el siguiente queda bien definida incluso si están pegados unos a otros.

```
if=while+else(true){print*&&}
```

```
1:0 IF if
1:2 ASSIGN =
1:3 WHILE while
1:8 PLUS +
1:9 ELSE else
1:13 LPAREN (
1:14 TRUE true
1:18 RPAREN )
1:19 LBRACE {
1:20 PRINT print
1:25 STAR *
1:26 AND &&
1:28 RBRACE }
2:0 TOKEN_EOF
```

La tercera (ut\_compound\_ops\_and\_comments) es la más cargada de casos límite, un comentario que empieza a la mitad de una línea (no solo al principio del archivo), tres diagonales seguidas

(///), donde el comentario tiene precedencia sobre un posible token **SLASH** previo, una cadena **===<=** donde se debe leer primero **==** como **EQUAL** y el **=** restante como **ASSIGN** suelto, seguido de **<** y **<=** igualmente distinguidos correctamente, y por último la palabra **EOF** escrita dentro de un comentario, para verificar que no se confunda con el fin real del archivo (el **TOKEN\_EOF** generado cae hasta la columna 24, es decir, hasta el final real del comentario, y no en la columna 21, que sería justo después de las dos diagonales).

```
intint ifif/// if if else
print===<=@while>=//EOF
```

```
1:0 IDENTIFIER intint
1:7 IDENTIFIER ifif
2:0 PRINT print
2:5 EQUAL ==
2:7 ASSIGN =
2:8 LESS <
2:9 LESS_EQUAL <=
2:11 ERROR @
2:12 WHILE while
2:17 GREATER_EQUAL >=
2:24 TOKEN_EOF
```

## Casos de error

La cuarta prueba (**ut\_error\_recovery\_mixed**) verifica que el analizador siga generando la secuencia completa de tokens después de encontrar varios caracteres inválidos seguidos, sin detener la ejecución del programa en ningún momento.

```
int x===0;
@
#
@
#
print(error)

1:0 INT int
1:4 IDENTIFIER x
1:5 EQUAL ==
1:7 ASSIGN =
1:8 INTEGER 0
1:9 SEMICOLON ;
2:0 ERROR @
3:0 ERROR #
4:0 ERROR @
5:0 ERROR #
6:0 PRINT print
6:5 LPAREN (
6:6 IDENTIFIER error
```

```
6:11 RPAREN )  
7:0 TOKEN_EOF
```

## Casos correctos

Los trece casos ya dados `tests/public/p2` (`make test-p2`) cubren, uno por uno, cada requisito puntual del enunciado por separado, identificadores, palabras reservadas, sensibilidad a mayúsculas y minúsculas, booleanos, cada operador compuesto, operadores incompletos, comentarios, comentarios al final del archivo sin salto de línea, la distinción entre `SLASH` y comentario, coincidencia más larga, recuperación de errores, un programa completo combinando todo lo anterior, y `CRLF`. Los trece pasan sin ningún `FAIL`, y junto con nuestras propias pruebas de casos límite y de error, nos dan bastante confianza de que la implementación cumple con el enunciado de la práctica de forma consistente y no solo para los ejemplos más obvios.

```
[code/] > make test-p2  
PASS: p01_identifiers  
PASS: p02_reserved_words  
PASS: p03_case_sensitive  
PASS: p04_booleans  
PASS: p05_compound_operators  
PASS: p06_incomplete_operators  
PASS: p07_comments  
PASS: p08_comment_at_eof  
PASS: p09_slash_and_comments  
PASS: p10_longest_match  
PASS: p11_error_recovery  
PASS: p12_complete_lexer  
PASS: p13_crlf  
----  
Total: 13 Pass: 13 Fail: 0
```

## Problemas encontrados

Nuestra primera versión de la estructura **Lexer** no usaba el mecanismo de *mirar antes de consumir* que terminamos implementando.

La idea original era mucho más parecida a lo que ya teníamos en la Práctica 1, una función `lexer_getc` que leía directamente del archivo con `fgetc` y, en la misma llamada, actualizaba línea y columna; y una función `lexer_ungetc` que, cuando una función auxiliar necesitaba espiar el siguiente carácter y resultaba no servirle (por ejemplo, un `=` que no estaba seguido de otro `=`), simplemente llamaba a `ungetc` sobre el archivo para regresarlo.

El problema es que `ungetc` solo le devuelve el carácter al `FILE*`; no revierte el avance de línea/-columna que `lexer_getc` ya había aplicado en el momento de leerlo. Como ese mismo carácter se volvía a leer en la siguiente vuelta del ciclo (y volvía a incrementar la columna), cada vez que una función auxiliar espiaba y regresaba un carácter, ese carácter terminaba contando el doble para la posición. El efecto se acumulaba y entre más operadores con anticipación aparecían en una misma línea, más se desviaba la columna reportada respecto de la esperada.

Esto es justo lo que se ve en la siguiente prueba, donde para el mismo archivo de prueba la columna reportada crece de forma consistente conforme se van encontrando más símbolos que requieren mirar hacia adelante:

```
[code/] > make test-cli
FAIL: p10_mixed_basic
--- tests/public/p1/expected/p10_mixed_basic.out 2026-09-06 00:45:49.020122446 -0600
+++ build/p10_mixed_basic.actual 2026-09-09 17:27:40.372148469 -0600
@@ -1,11 +1,11 @@
-1:0 LBRACE {
-2:4 INTEGER 10
-2:7 PLUS +
-2:9 INTEGER 20
-2:11 SEMICOLON ;
-3:4 INTEGER 30
-3:7 MINUS -
-3:9 INTEGER 5
-3:10 SEMICOLON ;
-4:0 RBRACE }
+1:1 LBRACE {
+2:5 INTEGER 10
+2:9 PLUS +
+2:11 INTEGER 20
+2:14 SEMICOLON ;
+3:5 INTEGER 30
+3:9 MINUS -
+3:11 INTEGER 5
+3:13 SEMICOLON ;
+4:1 RBRACE }
4:1 TOKEN_EOF
```

```
...
----
Total: 10 Pass: 2 Fail: 8
make: *** [Makefile:66: test-p1] Error 1
[code/] >
```

Un primer intento de solución fue agregar a la estructura `Lexer` campos de respaldo (`prev_line`, `prev_column`, `prev_last_was_cr`) para que `lexer_ungetc` pudiera restaurar exactamente el estado anterior a la última lectura. Esto sí resolvió el desfase creciente, pero expuso un segundo problema, más sutil, ahora todas las posiciones quedaban desplazadas por un valor constante de +1, sin importar el token. La causa era que `lexer_getc` seguía haciendo, en una sola llamada, tanto la lectura como el avance de columna; entonces, para cuando el ciclo principal capturaba `token_column = lex.column` (justo después de haber llamado a `lexer_getc` para obtener `c`), la columna ya reflejaba la posición del carácter *siguiente*, no la del carácter que acabábamos de leer.

En ese punto la estructura ya tenía ocho campos solo para poder llevar la cuenta de *antes* y *después* de cada lectura, y nos pareció que la complejidad no correspondía con lo que en el fondo queríamos resolver, solo necesitábamos poder decidir sin tener que deshacer nada después. Ahí fue cuando replanteamos el diseño desde la raíz, separando explícitamente dos operaciones que antes estaban fusionadas en una sola función, *mirar* (`lexer_check`, que no tiene ningún efecto secundario) y *consumir* (`lexer_advance`, la única operación que de verdad avanza línea, columna y trae el siguiente carácter del archivo).

Con `current` precargando siempre el carácter que todavía no se ha consumido, nunca necesitamos leer del archivo solo para decidir, y por lo tanto nunca necesitamos deshacer una lectura, la función `lexer_ungetc` desapareció por completo, y la estructura `Lexer` volvió a quedar con solo cinco campos, sin ningún campo de respaldo.

Como limitación conocida que queda pendiente, no implementamos ningún límite de longitud sobre identificadores o enteros más allá de la memoria disponible en el sistema (una decisión consciente, discutida en la sección anterior), y tampoco validamos todavía que un entero reconocido quepa en el rango de un tipo numérico concreto; ambas decisiones las dejamos para una etapa posterior del compilador.

## Conclusiones

Esta práctica nos dejó bastante claro que la robustez de un analizador léxico no depende únicamente de reconocer correctamente cada categoría de token por separado, sino de que el mecanismo de lectura sea consistente en todos los casos, incluyendo los que requieren mirar hacia adelante. Los errores que encontramos (el desfase creciente y luego el desfase constante) no venían de una regla léxica mal escrita, sino de una inconsistencia en cuándo se actualizaba la posición respecto de cuándo se leía o se decidía sobre un carácter; y ese tipo de error es particularmente traicionero porque el programa compila sin advertencias y produce tokens con el tipo correcto, solo que con la posición equivocada, lo cual fácilmente se nos podría haber pasado por alto de no haber tenido pruebas que verificaran columnas exactas.

Separar *mirar* de *consumir* como dos operaciones distintas, en vez de una sola función que hace ambas cosas a la vez, terminó siendo la simplificación clave, no solo resolvió los dos problemas de posición que tuvimos, sino que además redujo la estructura `Lexer` a menos campos que en nuestro primer intento de solución, y dejó cada función auxiliar (`scan_integer`, `scan_keyword`, `scan_operator`, `skip_comment_content`) más simple de leer, porque ninguna de ellas necesita ya preocuparse por deshacer una lectura.

Con todo esto, consideramos que llegamos a un analizador léxico completo para MINIC, reconocer todas las categorías de token pedidas en el enunciado, mantener un conteo correcto de línea y columna incluyendo casos de `\r`, `\n` y `\r\n`, recuperarse de errores léxicos sin detener la ejecución, y manejar lexemas de longitud arbitraria sin las limitaciones artificiales que sí tenía nuestra entrega de la Práctica 1.

Creemos que esta base modular (`Lexer`, `TextBuffer`, la tabla de `Keyword`) nos va a servir directamente como punto de partida para las siguientes etapas del compilador, en particular para el analizador sintáctico, que va a necesitar consumir exactamente esta misma secuencia de tokens.

## División del trabajo

Ambos integrantes participamos activamente en la elaboración de la práctica tanto del reporte como del código mismo, nos reunimos en el *Taller de Sistemas Simbólicos* del edificio *Tlahuizcalpan* para discutir sobre su elaboración, y posteriormente cada integrante fue incorporando su aportación a la práctica.

La compañera *María Fernanda* fue la de la idea de implementar la estructura `Lexer`, junto con sus nuevas funciones `lexer_init`, `lexer_advance`, por ende trabajó en mejorar su idea principal de `lexer_getc` y `lexer_ungetc` por la idea de *mirar antes de consumir* que la llevo a implementar `lexer_check`. Fue además quien implementó la función `scan_integer` como concepto para la función `scan_keyword`, con el uso del buffer de la Práctica 1.

El compañero *Manuel Ianluck* fue el de la idea de modularizar `lexer_scan`, para que cada caso mande llamar su respectiva función, por lo que creó la estructura `Keyword` y su implementación a partir de un pseudo hashmap. Además mejoró las funciones iniciales de su compañera `scan_integer` y `scan_keyword` para que utilizaran un buffer dinámico con lo echo en una actividad de clase, y justo el viernes 11 de septiembre se habló en el laboratorio sobre esta característica



ya implementada por nosotros.

## Uso de herramientas de Inteligencia Artificial

Para la realización de esta práctica no se llevó a cabo ninguna utilización de inteligencia artificial de ningún tipo, pura y dura inteligencia natural.

## References

- [1] ppreference.com. (n.d.). C language. Disponible en <https://en.cppreference.com/c/language>
- [2] ppreference.com. (n.d.). C11. Disponible en <https://en.cppreference.com/c/11>
- [3] ppreference.com. (n.d.). Strings library, strchr. Disponible en <https://en.cppreference.com/c/string/byte/strchr>
- [4] ppreference.com. (n.d.). Strings library, isalpha. Disponible en <https://en.cppreference.com/c/string/byte/isalpha>
- [5] ppreference.com. (n.d.). Strings library, isalnum. Disponible en <https://en.cppreference.com/c/string/byte/isalnum>